FlowSight

AI-powered V2X Traffic Intelligence

Technical Documentation

Full-Stack Engineering Assessment

June 2026

Contents

1 Objective

FlowSight is a web application that presents traffic data through interactive charts and an interactive congestion map. It satisfies four functional requirements — an interactive *country-wise traffic* chart and an interactive *vehicle-type distribution* chart on the front end; a REST API that serves the data; a relational database that stores and allows updates to the data; and a documented strategy for scaling from 5 to 50 to 500 requests per second (RPS).

Beyond the brief, FlowSight adds a number of features that make the product feel like a real operations dashboard: server-side filtering with a date range; real-time updates over WebSockets; KPI summary cards with a live trend indicator; an interactive *Greater Vancouver* congestion map with a time-travel slider, incident impact radii and border-crossing wait times; an overall traffic-status badge; a top-congested-routes panel; and live notification toasts. It also ships with containerisation, a CI pipeline, and unit tests on both ends.

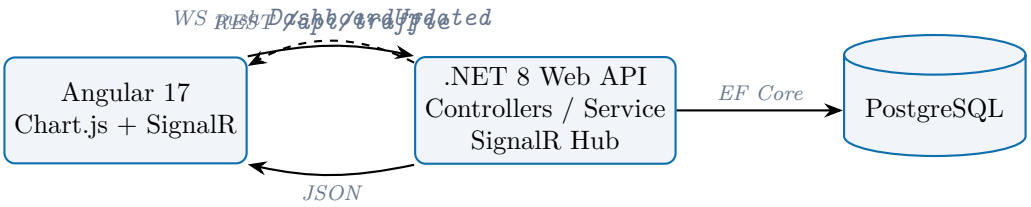
2 Technology Stack

Layer	Technology
Front end	Angular 17 (standalone components), Chart.js 4
Mapping	Leaflet 1.9 + OpenStreetMap tiles
Real-time client	`@microsoft/signalr`
Back end	.NET 8 ASP.NET Core Web API
Data access	Entity Framework Core 8 (Npgsql provider)
Real-time server	ASP.NET Core SignalR
Database	PostgreSQL 16
Containerisation	Docker, Docker Compose
CI/CD	GitHub Actions
Testing	xUnit (back end), Jasmine + Karma (front end)

The stack was chosen to match the requested .NET + Angular pairing while staying close to the assessment's preferences (a relational database; a well-supported framework). PostgreSQL is paired with EF Core via the mature Npgsql provider, giving migrations, LINQ-to-SQL translation, and strong typing.

3 System Architecture

The system follows a conventional three-tier design with an additional real-time channel running alongside the request/response path.



3.1 Request flow

1. On load, the Angular app calls `GET /api/traffic/filters` to populate the country and vehicle-type dropdowns, and `GET /api/traffic` for the initial chart data.
2. The controller delegates to `TrafficService`, which builds two aggregate queries (grouped sums) and returns a compact payload.

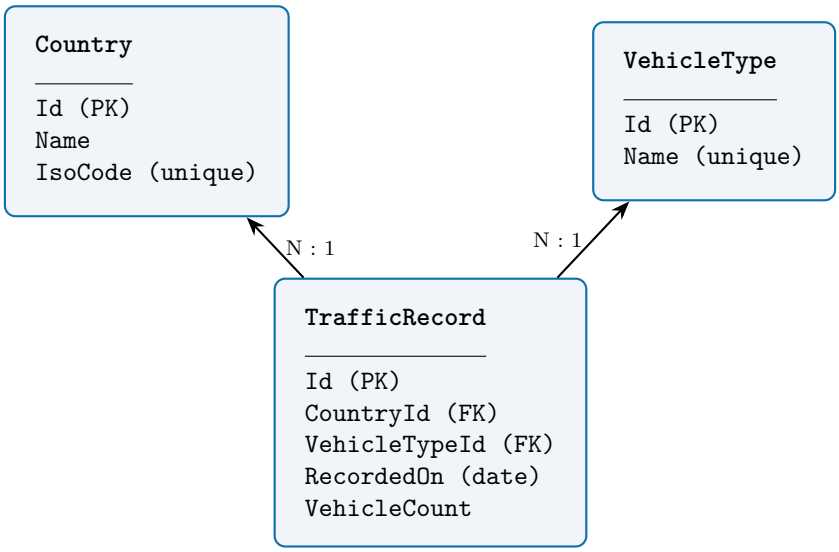
3. All grouping and filtering is executed in SQL by PostgreSQL, so the response carries only the aggregated points the charts need.

3.2 Real-time flow

1. The browser opens a SignalR connection to `/hubs/traffic`.
2. When a new record is posted to `POST /api/traffic`, the service persists it, recomputes the dashboard, and the controller broadcasts it to all clients via `IHubContext`.
3. Each client receives the `DashboardUpdated` event and re-renders both charts without a page reload.

4 Data Model

The domain splits into two areas. The **analytics** area has two reference tables (`Countries`, `VehicleTypes`) and one fact table (`TrafficRecords`) that drive the dashboard charts. The **live-map** area adds four tables — `RoadSegments`, `CongestionReadings`, `Incidents` and `BorderWaits` — that drive the Greater Vancouver map.



4.1 Tables

Column	Type	Notes
Countries		
Id	integer (identity)	primary key
Name	varchar(100)	required
IsoCode	varchar(3)	unique, e.g. `ARE`
VehicleTypes		
Id	integer (identity)	primary key
Name	varchar(50)	unique, e.g. `Car`
TrafficRecords		
Id	integer (identity)	primary key
CountryId	integer (FK)	cascade delete
VehicleTypeId	integer (FK)	cascade delete
RecordedOn	date	observation date

Column	Type	Notes
VehicleCount	integer	aggregated count
RoadSegments		
Id	integer (identity)	primary key
Name	varchar(120)	corridor name
PathJson	text	JSON `[lat,lng]` polyline
Capacity	integer	reference throughput (veh/hr)
CongestionReadings		
Id	integer (identity)	primary key
RoadSegmentId	integer (FK)	cascade delete
HourBucket	integer	6, 12, 18 or 22
Volume	integer	observed vehicles/hr
Incidents		
Id	integer (identity)	primary key
Title, Description	varchar	incident detail
Lat, Lng	double precision	location
RadiusMeters	integer	impact radius
Severity	varchar(20)	Low / Medium / High
HourBucket	integer	active hour
BorderWaits		
Id	integer (identity)	primary key
Name	varchar(80)	crossing name
Lat, Lng	double precision	location
HourBucket	integer	6, 12, 18 or 22
WaitMinutes	integer	southbound wait

4.2 Indexes

A composite index on `(CountryId, VehicleTypeId, RecordedOn)` and a single-column index on `RecordedOn` support the filtered aggregation and date-range queries. The map tables are indexed on `(RoadSegmentId, HourBucket)` and on `HourBucket` so a single-hour lookup is cheap. Unique indexes on `IsoCode` and `VehicleType.Name` guarantee reference-data integrity.

4.3 Seed data

The seeder is idempotent: it inserts each reference row only if it is missing and backfills traffic records for any country that has none, so an already-seeded database picks up new entries (such as Canada) on the next run without a reset. All values are generated deterministically (no randomness), so every environment produces identical charts and maps.

Listing 1: Seeded data

```
Countries : Canada, United States, United Arab Emirates,
               Germany, India, Japan, United Kingdom
VehicleTypes : Car, Truck, Bus, Motorcycle, Bicycle
Snapshot dates: 2026-06-01, 2026-06-05, 2026-06-09
Map (Greater Vancouver):
  8 road segments x 4 hour buckets of congestion readings,
  3 incidents, 2 border crossings (Peace Arch, Aldergrove).
```

5 Backend API

5.1 Endpoints

Method	Route	Purpose
GET	`/api/traffic`	Aggregated dashboard data. Optional query parameters: `countryId`, `vehicleTypeId`, `from`, `to`.
GET	`/api/traffic/filters`	Lists countries and vehicle types for the filter controls.
POST	`/api/traffic`	Ingests one traffic record and broadcasts the refreshed dashboard.
GET	`/api/map`	Live-map data for an hour bucket (`?hour=6|12|18|22`): road segments with congestion level and estimated delay, incidents, and border-crossing waits.
WS	`/hubs/traffic`	SignalR hub; emits `DashboardUpdated`.

5.2 Response shape

Listing 2: `GET /api/traffic` response

```
{
  "countryTraffic": [ { "label": "United States", "value": 5120 }, ... ],
  "vehicleDistribution": [ { "label": "Car", "value": 9800 }, ... ],
  "totalVehicles": 24310,
  "trendPercent": 3.2, // change vs the previous snapshot date
  "hasTrend": true
}
```

Listing 3: `GET /api/map?hour=18` response (excerpt)

```
{
  "hour": 18,
  "segments": [ { "name": "Highway 1 - Port Mann Bridge",
                  "path": [[49.21,-122.80], ...],
                  "volume": 6300, "capacity": 6000,
                  "level": "heavy", "delayMinutes": 14 }, ... ],
  "incidents": [ { "title": "Multi-vehicle collision",
                   "lat": 49.22, "lng": -122.79,
                   "radiusMeters": 1200, "severity": "High" } ],
  "borders": [ { "name": "Peace Arch (Douglas)",
                   "waitMinutes": 50, "level": "heavy" } ]
}
```

5.3 Aggregation logic

The service applies the optional filters, then groups the fact table twice — once by country and once by vehicle type — summing `VehicleCount`. Both queries run `AsNoTracking` for read performance and translate to a single SQL `GROUP BY` each.

Listing 4: Core aggregation (C#, simplified)

```
var countryTraffic = await filtered
```

```
    .GroupBy(r => r.Country.Name)
    .Select(g => new ChartPoint(g.Key, g.Sum(r => (long)r.VehicleCount)))
    .OrderByDescending(p => p.Value)
    .ToListAsync();
```

5.4 Updating data

Records are created through `POST /api/traffic`. Because the same service layer is used by both reads and writes, a write is immediately reflected in the next aggregation and pushed live to every connected client.

6 Frontend

The Angular application is built from standalone components:

- `FilterBarComponent` — country, vehicle and date-range controls that emit a filter object on *Apply*.
- `KpiCardsComponent` — summary statistic cards (total vehicles with a trend arrow, counts, top country, top vehicle).
- `CountryChartComponent` — a Chart.js chart with a bar/line toggle.
- `VehicleChartComponent` — a Chart.js doughnut chart with percentage tooltips.
- `LiveFeedComponent` — a connection indicator and a button that posts a sample event to demonstrate real-time updates.
- `LiveConditionsComponent` — the traffic-status badge and top-congested-routes list (driven by `/api/map`).
- `MapDashboardComponent` — the Leaflet map, time-travel slider and incident overlays.
- `BorderWidgetComponent` — the border-crossing wait-time card.
- `AppComponent` — composes the above behind a Dashboard/Live-Map tab switch, owns the current filter and toast notifications, and subscribes to both the REST service and the SignalR stream.

Three injectable services isolate I/O: `TrafficService` and `MapService` (REST) and `RealtimeService` (SignalR). The API base URL is supplied through an injection token, so it can be swapped per environment without touching component code. The layout is responsive — charts and panels sit side by side on wide screens and stack on narrow ones.

7 Real-Time Updates

SignalR provides a managed abstraction over WebSockets with automatic fallback and reconnection. The server exposes an empty `TrafficHub`; broadcasting is driven from the controller through `IHubContext<TrafficHub>`, which keeps the hub itself free of business logic.

Listing 5: Broadcast on ingest (C#)

```
var record = await _service.AddRecordAsync(dto, ct);
var dashboard = await _service.GetDashboardAsync(new TrafficQuery(), ct);
await _hub.Clients.All.SendAsync("DashboardUpdated", dashboard, ct);
```

On the client, `RealtimeService` exposes the pushes as an RxJS `Observable`, which `AppComponent` subscribes to and uses to refresh the charts.

8 Live Traffic Map

The *Live Map* tab renders an interactive Leaflet map of Greater Vancouver. It is intentionally a different zoom level from the dashboard: the dashboard is a country-level overview, while the map is a city-level detail view of one location, giving an overview-then-drill-down narrative.

8.1 Congestion colouring

Each `RoadSegment` is drawn as a polyline coloured by its congestion level. The level is derived server-side from the volume-to-capacity ratio: below 50 % is *low* (green), below 85 % is *moderate* (amber), and above that is *heavy* (red). The API also returns an estimated `delayMinutes` per segment, scaled from the same ratio.

8.2 Time-travel slider

A slider steps through four hour buckets (6 am, 12 pm, 6 pm, 10 pm). Each change re-fetches `/api/map?hour=...` and redraws the overlays, so the user can watch congestion build through the morning and evening peaks. Seeded volumes follow a realistic daily curve (rush at 6 and 18, quiet overnight).

8.3 Incidents and border crossings

Incidents are drawn as a translucent circle (the impact radius) with a clickable centre marker. A toggle hides or shows them. The `BorderWidgetComponent` lists southbound wait times for the Peace Arch and Aldergrove crossings, colour coded by severity, updating with the slider.

9 Dashboard Intelligence

Several touches make the dashboard read like a live operations console; all figures are computed from real data rather than hard-coded.

- **KPI trend indicator.** The total-vehicles card shows a green/red arrow and percentage. The backend computes this as the change in total volume between the two most recent snapshot dates in scope (`trendPercent`), and omits it when fewer than two dates are available.
- **Traffic-status badge.** `LiveConditionsComponent` averages the current segment load into a four-level status — Normal (green), Moderate (yellow), Busy (orange), Congested (red).
- **Top congested routes.** The same panel ranks road segments and border crossings by delay/wait minutes and lists the worst offenders.
- **Live toasts.** When a SignalR update arrives, the client compares the new total against the previous one and shows a toast with the real delta (e.g. "+287 vehicles detected (+0.6%)").

10 Scalability: 5 to 50 to 500 RPS

The API is stateless, which is the key property that makes horizontal scaling possible. The plan below grows the system in three stages.

10.1 5 RPS — single instance

A single API process and a single PostgreSQL instance comfortably serve this load. The relevant work here is correctness and baseline efficiency: indexed queries (already in place), EF Core connection pooling, and `AsNoTracking` reads. Vertical headroom is ample.

10.2 50 RPS — scale out and cache

- **Multiple API replicas** behind a load balancer. Because the API holds no per-request state, replicas are interchangeable.
- **Redis response cache** on the aggregation endpoint. Dashboard data changes far less often than it is read, so short-lived caching (with cache invalidation on ingest) removes most database load.
- **SignalR Redis backplane** so a push from any replica reaches clients connected to any other replica.
- **Connection-pool tuning** and PgBouncer in front of PostgreSQL.

10.3 500 RPS — elastic and distributed

- **Auto-scaling** of API pods on Kubernetes (Horizontal Pod Autoscaler keyed on CPU/RPS).
- **Read replicas** for PostgreSQL; the read-heavy aggregation traffic is routed to replicas while writes go to the primary.
- **Materialized views** (or a pre-aggregated summary table refreshed on ingest) so the hottest queries become simple lookups.
- **CDN** for the static Angular bundle, removing front-end delivery from the origin entirely.
- **Message queue** (e.g. Kafka/RabbitMQ) in front of ingestion to absorb write spikes and decouple producers from the database.
- **Observability** — metrics, tracing and autoscaling alarms to keep the system within its latency budget.

	5 RPS	**50 RPS**	**500 RPS**
API	1 instance	N replicas + LB	autoscaled pods
Cache	none	Redis	Redis + materialized views
Database	single	single + PgBouncer	primary + read replicas
Real-time	in-process	Redis backplane	Redis backplane
Static	origin	origin	CDN
Ingestion	direct	direct	message queue

11 Testing

11.1 Backend (xUnit)

`TrafficServiceTests` uses the EF Core in-memory provider to verify the aggregation logic in isolation: unfiltered totals, filtering by country, date-range exclusion, and that a newly added record is counted. Each test runs against a fresh database instance.

11.2 Frontend (Jasmine + Karma)

`TrafficService` is tested with `HttpTestingController` to confirm the correct URLs and query parameters are produced. `FilterBarComponent` is tested for its emit-on-apply and reset behaviour.

12 Deployment

12.1 Docker Compose

A single `docker compose up -build` starts three services: PostgreSQL, the API (which migrates and seeds on startup), and the Angular app served by nginx. A health check on the database gates the API start-up so migrations never race an unready database.

Service	Image / build	Host port
db	`postgres:16-alpine`	5432
api	`backend/Dockerfile`	5000 → 8080
frontend	`frontend/Dockerfile`	8081 → 80

12.2 CI pipeline (GitHub Actions)

On every push and pull request to `main`, two parallel jobs run. The back-end job restores, builds and runs `dotnet test`; the front-end job installs dependencies, runs the Jasmine suite headless, and produces a production build. A green pipeline therefore proves both halves compile and pass their tests.

13 Running the Project

With Docker:

```
docker compose up --build
# Frontend -> http://localhost:8081
# API/Swagger -> http://localhost:5000/swagger
```

Locally:

```
# Backend
cd backend && dotnet run --project FlowSight.Api

# Frontend
cd frontend/flowsight-web && npm install && npm start
```

14 Design Decisions and Trade-offs

- **Aggregate in the database, not the client.** Grouping and summing in SQL keeps payloads tiny and pushes work to the layer best suited for it.
- **Stateless API.** The single most important decision for scalability; it makes every later scaling step (replicas, autoscaling) straightforward.
- **Broadcast from the controller, not the hub.** Keeps the hub thin and the business logic in one place (the service layer).
- **Injection token for the API URL.** Environment configuration without code changes.
- **Minimal, deterministic seed data.** Enough to exercise both charts and every filter, reproducible across environments, with no bloat.
- **Honest, computed metrics.** Trends, status and delays are derived from the data, not invented, so the figures hold up under inspection.

This document describes the structure and reasoning behind the FlowSight codebase included in the same repository.